

Toy ASR

Ellison Murray

August 5, 2026

Introduction

This document provides a step-by-step walkthrough of a toy Automatic Speech Recognition (ASR) model. Through layer-by-layer visual diagrams and structural descriptions, it details how audio data is transformed and propagated through the network to generate text predictions. This serves as an accessible conceptual introduction for readers new to ASR or machine learning in general.

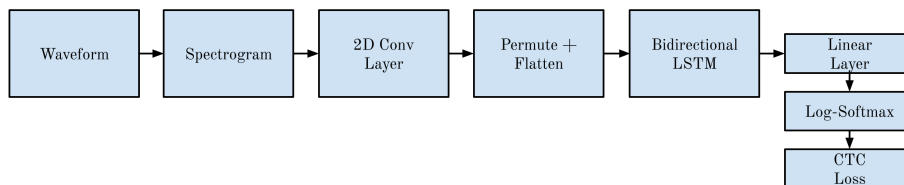


Figure 1: Block Diagram of Toy ASR Model

Dataset

This project uses single English speech commands from the Google Speech Commands Dataset. Audio is processed as 16-bit PCM at 16 kHz.

Model

Front End

The input audio is represented as a $T \times 80$ tensor corresponding to log-mel spectrogram features across 80 frequency channels.

We permute the second layer's output:

$$(\text{Batch}, \text{Channel}, \text{Frequency}, \text{Time}) \rightarrow (\text{Batch}, \text{Time}, \text{Channel}, \text{Frequency}).$$

Then we reshape[cite: 1]:

$$(\text{Batch}, \text{Time}, \text{Channel}, \text{Frequency}) \rightarrow (\text{Batch}, \text{Time}, \text{Channel} \times \text{Frequency}).$$

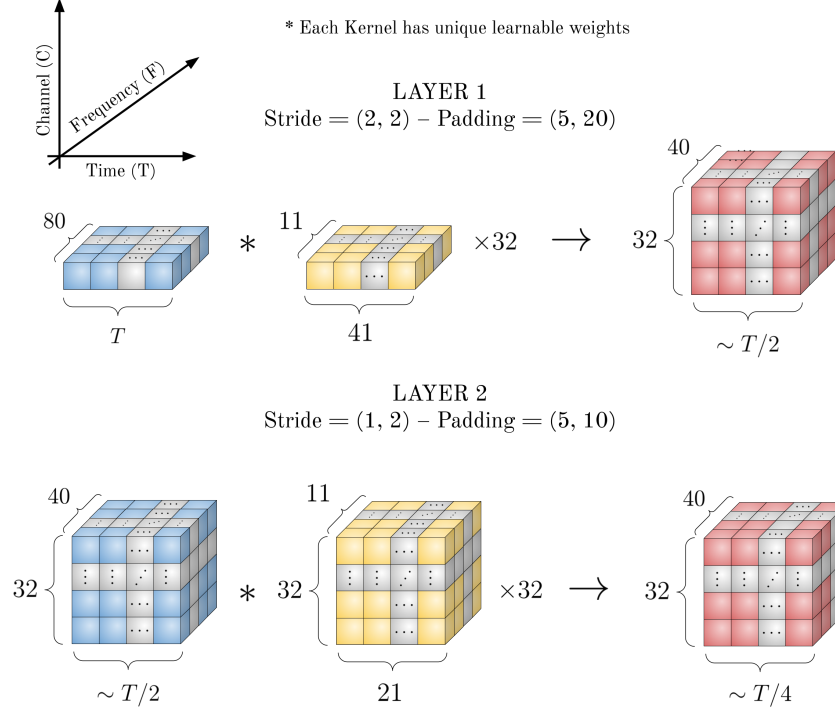


Figure 2: Structural Tensor Mechanics of the Model 2 Front-End.

Bidirectional Long Short-Term Memory Model

The dimension of the hidden state h_t is 128.

Refer to Introduction to Long Short-Term Memory for a more in-depth explanation of this architecture.

Our model considers a two-layer LSTM architecture[cite: 1]. The equations below provide a crude simplification of the model, given a recurrent function $f(\cdot)$:

$$h_t^{(1)} = f(x_t, h_{t-1}^{(1)}), \quad h_t^{(2)} = f(h_t^{(1)}, h_{t-1}^{(2)}).$$

Because we use a bidirectional LSTM, for each time step t the hidden state dimension doubles (combining forward and backward passes):

$$(\text{Batch}, \text{Time}, 2 \times \text{hidden_dim}) \rightarrow (\text{Batch}, \text{Time}, \text{vocab_size})$$

Before performing CTC loss, we pass our sequence of hidden states through a linear layer to project to the vocabulary dimension. This tensor is then passed through a Log-Softmax function, converting it into a PMF—the discrete probabilities of each character occurring at time step t .

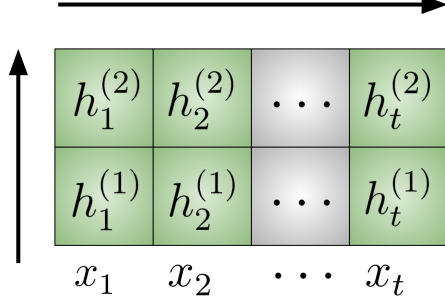


Figure 3: 2-Layer LSTM Structural Mapping

Connectionist Temporal Classifier

When performing CTC loss, we have a known target, and because the actual output sequence is often longer than the target sequence, repeats and spaces can exist. CTC introduces a blank character ϵ . Refer to Connectionist Temporal Classification for a better explanation of how this handles repeated characters via collapsing.

$$\mathcal{L} = -\log P(Y = \text{Target} \mid X)$$

Given the input sequence with character PMFs, CTC evaluates the probability of the target string occurring. The higher the probability, the smaller the loss, allowing the model to learn.

CTC computes this by considering all valid alignments of the target within the output sequence length[cite: 1]. Let π denote an alignment and $\mathcal{A}(\text{Target})$ denote the set of valid alignments:

$$P(Y = \text{Target} \mid X) = \sum_{\pi \in \mathcal{A}(\text{Target})} P(\pi \mid X)$$

Because we have the PMF at each time step within the sequence, CTC can directly compute $P(\pi \mid X)$.

Each input vector x_t , for an all-lowercase alphabet plus the blank token, has length $26 + 1 = 27$: $x_t = [p_a, p_b, \dots, p_z, p_\epsilon]^T$, where $\|x_t\|_1 = 1$.

Example. Suppose Target = abc. For an output sequence of length four, valid alignments include $ab\epsilon c$ and ϵabc . Let $\pi = ab\epsilon c$:

$$P(\pi \mid X) = P(y_1 = a \mid x_1)P(y_2 = b \mid x_2)P(y_3 = \epsilon \mid x_3)P(y_4 = c \mid x_4)$$